
Federated Schedule: Behavioral-Aware System Heterogeneity in Federated Learning

Carter Hand¹

¹Auburn University,
cbh0098@auburn.edu

Abstract

Standard Federated Learning defines client availability through binary hardware states such as connected and idle. This hardware-centric approach ignores behavioral heterogeneity, which refers to the rhythmic and predictable patterns of human-device interaction. When users unplug unpredictably, Zombie Training cycles occur. In these cycles, local computation is discarded, which wastes energy and slows global model convergence. We propose Federated Schedule, a proactive scheduling layer that models user-specific charging windows to adaptively scale workloads. By predicting availability and matching task complexity to the predicted time window, our system eliminates inefficient cycles and improves the accuracy per joule of the training process.

0.1 Background

Federated Learning has emerged as the leading paradigm for privacy-preserving distributed training in mobile and edge computing environments [1]. By moving the model to the data rather than the data to a central server, Federated Learning enables the training of high-performance models on sensitive information. This is particularly vital in specialized fields such as medical imaging where data privacy and trustworthiness are paramount [1].

0.2 Motivation

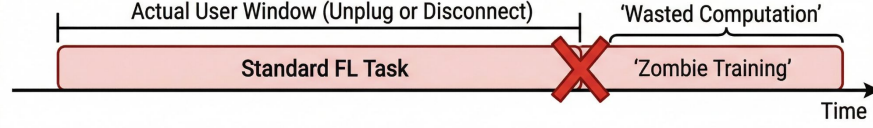
Despite the success of existing frameworks, current protocols rely on a simplistic binary definition of device availability. Most systems assume that any device which is plugged in and idle is ready for a full training round. This assumption fails to account for human behavior. In the real world, the same hardware state can represent vastly different levels of availability. A device charging overnight provides a stable window of several hours, while a morning charge may only last a few minutes before the user unplugs for a commute. Treating these sessions as identical leads to ****Zombie Training****. In these cycles, a device starts a training task but is disconnected before completion, causing the server to discard the update and wasting the energy spent by the device.

0.3 Challenges

The primary challenge in this research is addressing behavioral heterogeneity. While significant work exists for system heterogeneity and data heterogeneity, modeling the unpredictable nature of human-device interaction is difficult. Unlike hardware specifications, behavior is rhythmic but stochastic. Predicting a specific charging window requires local history analysis that does not compromise user privacy. Furthermore, the system must be able to adapt the training workload in real-time to fit these predicted windows without destabilizing the global model convergence.

"Before vs. After" Comparison: The Zombie Training Problem vs. Federated Schedule Solution

A. Scenario A: Zombie Training – The Problem



B. Scenario B: Federated Schedule – The Solution

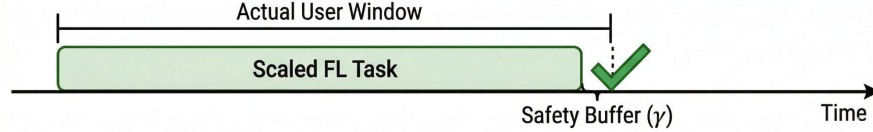


Figure 1: A comparison of training cycles. Top: Standard FL results in "Zombie Training" when the task exceeds user availability. Bottom: Federated Schedule resizes the task to fit the predicted window, ensuring successful completion.

1 Related Work

1.1 System and Model Heterogeneity

Existing optimization methods such as FedProx attempt to mitigate system heterogeneity by allowing for partial updates from slow devices [2]. While effective for stragglers, these methods do not proactively predict when a client will disconnect entirely. Conversely, model-heterogeneous frameworks like HeteroFL [3] and FedRox [4] provide the technical machinery to distribute sub-models of varying sizes. Federated Schedule builds upon these techniques by using predicted time as the primary trigger for model scaling.

1.2 Availability-Aware Systems

Research into client selection has focused on identifying which clients are most likely to provide high-quality updates. Benchmarks like FedScale have quantified the massive performance loss associated with high client dropout rates [5]. Our work moves beyond mere selection by proposing workload adaptation, which allows even highly mobile clients to participate in training through right-sized tasks.

2 Methodology

2.1 Charging Window Prediction

The first component is the local behavior predictor. Each client maintains a private log of charging events to compute a probability density [4]. We formalize the predicted window T_i^{pred} as the expected remaining charging time given the client's local historical patterns [3]:

$$T_i^{pred} = \mathbb{E}[\text{remaining charging time} \mid \text{history}]$$

To implement this, we utilize a histogram-based estimate of past sessions to forecast the likely unplug time [5].

2.2 Adaptive Workload Selection

When a client is selected for a round, the server utilizes the predicted window to choose a workload size. This involves selecting a sub-model M_k and a local epoch count E . The selection logic ensures

that the estimated training time is strictly less than the predicted availability window. A safety buffer is applied to this calculation to protect against the inherent variance in human behavior. This ensures the client completes the local stochastic gradient descent and uploads the results before the user interrupts the session.

2.3 Reliability-Weighted Aggregation

To protect the global model from the variance introduced by different sub-model sizes, we adjust the aggregation weight α_i for client i . We utilize a weighting function where the contribution weight is proportional to the predicted stability of the client [2]:

$$\alpha_i = \frac{n_i \cdot \phi(T_i^{pred})}{\sum_{j=1}^N n_j \cdot \phi(T_j^{pred})}$$

This approach prioritizes updates from stable sessions while still including data from highly mobile users [1].

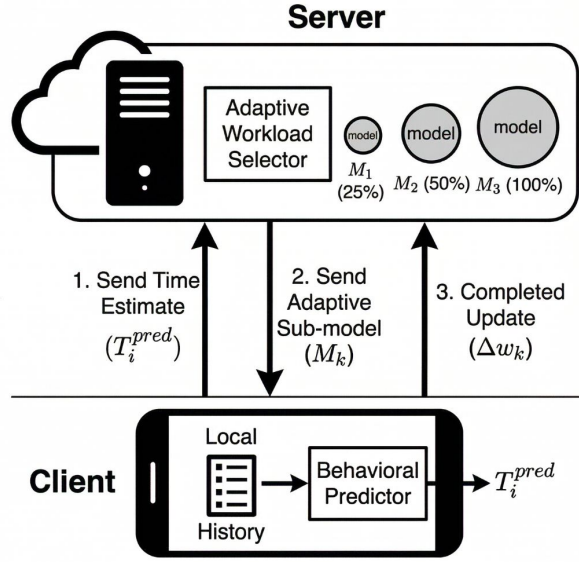


Figure 2: System Architecture of Federated Schedule. The proactive loop begins with the client behavioral prediction T_i^{pred} , which is then followed by the adaptive workload selection M_k performed by the server.

The core of Federated Schedule is a proactive feedback loop between the client and the server. Unlike reactive systems that wait for a client to fail, our system predicts availability before the round begins to eliminate Zombie Training cycles.

3 Experimental Plan

3.1 Simulation Setup

We will utilize the FedScale framework to simulate a population of 1,000 clients [5]. To evaluate the impact of human behavior, we will synthesize behavioral profiles ranging from stable overnight chargers to high-churn irregular users. This allows us to test the robustness of the predictor across different usage rhythms.

3.2 Baselines and Data

Our method will be compared against the standard FedAvg protocol [1] and the heterogeneity-aware FedProx [2]. We will conduct evaluations on the CIFAR-10 image dataset [6] and the Shakespeare

language modeling dataset. These benchmarks provide a comprehensive look at how our system handles both computer vision and natural language processing tasks.

3.3 Evaluation Metrics

The primary metric for success is **Training Goodput**. We formalize Goodput as the ratio of successfully completed updates to the total number of updates initiated by the server [5]:

$$\text{Goodput} = \frac{\text{Completed Updates}}{\text{Started Updates}}$$

This provides a direct measure of system efficiency and the reduction of "Zombie Training" cycles [2].

3.4 Hyperparameters

Table 1: Key simulation and training hyperparameters.

Parameter	Value
Global Communication Rounds	200
Clients per Round	10
Target Safety Buffer (γ)	0.8
Confidence Threshold (τ)	0.9
Initial Learning Rate	0.01

γ is empirically chosen to provide a conservative buffer against prediction uncertainty.

4 Conclusion

Federated Schedule addresses the root cause of training inefficiency in mobile environments by shifting from a reactive to a proactive paradigm. By understanding the behavioral patterns behind the device hardware, we can eliminate the wasted energy of Zombie Training. This research aims to prove that behavior-aware scheduling makes distributed learning more efficient and sustainable. Our future work will focus on integrating reinforcement learning to improve prediction accuracy and exploring differential privacy to further secure the local behavior models.

References

- [1] Brendan McMahan et al. Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 2017.
- [2] Tian Li et al. Federated optimization in heterogeneous networks. *MLSys*, 2020.
- [3] Enmao Diao et al. Heterofl: Computation and communication efficient federated learning for heterogeneous clients. In *ICLR*, 2020.
- [4] Samiul Alam et al. Fedrolex: Model-heterogeneous federated learning with rolling sub-model extraction. In *NeurIPS*, 2022.
- [5] Fan Lai et al. Fedscale: Benchmarking model and system performance of federated learning at scale. In *ICML*, 2021.
- [6] Alex Krizhevsky and Hinton. Learning multiple layers of features from tiny images. <http://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>, 2009.